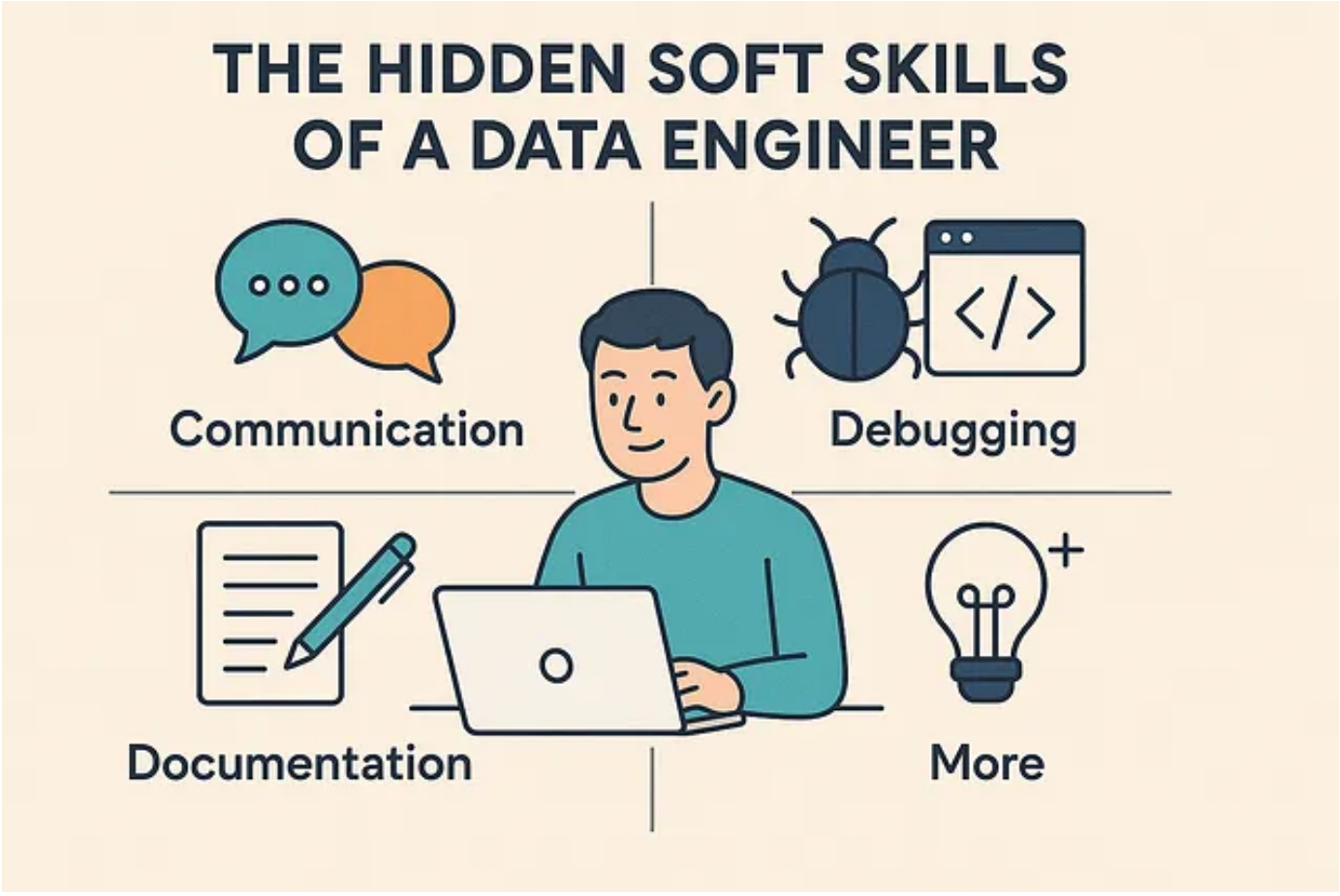
The Hidden Soft Skills of a Data Engineer: Communication, Debugging, Documentation, and More

Introduction

Vijay Ashley Rodrigues



As a data engineer, you're often deep in the technical weeds — designing ETL pipelines, managing databases, and working with massive datasets. But what really sets a good

data engineer apart from a great one isn't just the tools they use or the code they write. It's the **soft skills** they bring to the table. These skills, while often less visible, are crucial in ensuring your work is effective, scalable, and understood by others.

In this post, I'll dive into some of the soft skills that I've found most valuable in my work as a data engineer and share a few real-life examples where these skills have made a difference.

Communication: The First and Last Step of Every Task

As technical as data engineering can be, **communication** is where it all starts and ends. Whether it's clarifying project requirements or explaining data findings to a non-technical team, the ability to communicate effectively can make all the difference. I've learned that taking the time to ask the right questions upfront, rather than assuming things, saves countless hours later.

For example, I once worked on a project where I was tasked with extracting data on "active users." At first glance, it seemed straightforward. But after a quick discussion with the product team, it turned out they meant "users who logged in within the last month." Without this clarification, I would have delivered a report based on a completely different interpretation of the request.

Debugging: The Mindset That Solves More Than Errors

Debugging is more than just finding and fixing bugs — it's about having the right mindset to approach problems systematically. In data engineering, issues can arise in any part of the pipeline, and knowing how to troubleshoot effectively is essential. Over the years, I've realized that a calm, methodical approach is key to resolving issues without wasting time on wild guesses.

For example, during one project, I was dealing with a data pipeline that kept failing after a schema change in the source database. Instead of panicking or guessing the problem, I took a step back and asked myself, "What changed in the schema? Was there a new column added? Was the data type altered?" By breaking the problem down logically, I quickly pinpointed the root cause and was able to fix the issue efficiently.

Documentation: Helping Future You (and Your Team)

I can't stress enough how valuable good **documentation** is. It's something I've had to learn the hard way — good documentation not only helps others understand your work but also saves you time down the road when you have to revisit old projects. I've found that well-commented code, clear variable names, and concise explanations of complex logic go a long way in making projects more maintainable.

For instance, when I was working on an ETL pipeline a while ago, I made sure to document the reasoning behind certain design decisions. A few months later, a colleague needed to update the pipeline, and having that context made the process much easier. Without it, they would have had to reverse-engineer my logic from scratch.

Navigating Scope Changes: Flexibility with Clarity

If you've been in the industry for long, you know that **scope changes** are inevitable. Business priorities shift, new requirements emerge, and deadlines get compressed. The ability to adapt without compromising the quality of your work is a key soft skill that I've learned over time. But just as important is the ability to clearly communicate the impact of these changes.

For example, I once worked on a project where I had to convert scripts from one language to SQL. The initial plan

was simple, but as I started analyzing the existing scripts, I realized they were much more complex than anticipated. The logic in the original scripts was intricate and had to be carefully deciphered in order to translate it accurately into SQL. This led to some delays, but I made sure to communicate the challenges clearly to the team, explaining the additional time required for understanding and reworking the logic. Instead of a rushed, half-baked solution, we ended up with a well-documented, properly converted set of SQL scripts that worked seamlessly in the new environment.

Cross-Team Collaboration: Speaking *Their* Language

As a data engineer, you often work with teams outside your direct area of expertise, like product managers, analysts, and DevOps. **Cross-team collaboration** is essential in making sure your work aligns with the bigger picture. I've found that the best way to collaborate with non-technical teams is by speaking their language, understanding their goals, and translating technical requirements into terms they can relate to.

For example, I had a situation where an analyst requested a dataset, but the requirements were unclear. Instead of assuming I knew exactly what they needed, I took the time to have a discussion. It turned out that the metrics they were requesting weren't clearly defined. By getting the details

straight, I was able to deliver a dataset that better met their needs, and that saved both of us time in the long run.

Final Thoughts

While technical skills are obviously critical in data engineering, I've come to realize that soft skills like **communication**, **debugging**, **documentation**, handling **scope changes**, and effective **cross-team collaboration** are just as important. These skills help us solve problems more efficiently, ensure that our work is scalable and maintainable, and ultimately make us better teammates.

By honing these hidden soft skills, you'll not only become a better data engineer but also a more valuable member of any team. It's these skills that help you navigate the challenges of real-world projects and contribute to the success of your team.